BIOINFORMÁTICA

Montagem de Genoma de Novo com Dados Illumina

Versão	Data	Responsável
v1.0	25/06/2026	Joyce R. Prado

Objetivo

Este material foi escrito para alunos sem experiência prévia em montagem de genomas ou em uso de clusters de alto desempenho (HPC), com base em um estudo de caso com o genoma de *Thrichomys pachyurus*. Cada etapa explica o que o comando faz, por que ele é necessário, e o que observar no resultado antes de seguir adiante.

Sumário

1. Visão Geral do Processo 4

1.1 Conceitos Essenciais Antes de Começar 5

1.2 Antes de Começar: O Que Você Precisa Ter 5

2. Preparando o Ambiente de Trabalho 6

2.1 Introdução ao Conda 6

2.2 Carregando o Conda no Cluster 6

2.3 Criando o Ambiente do Projeto 6

2.4 Aceitando os Termos de Uso dos Canais (se solicitado) 7

2.5 Ativando o Ambiente 7

2.6 Verificando se Tudo Foi Instalado Corretamente 7

2.7 Organizando os Diretórios do Projeto 8

3. Transferindo os Arquivos de Sequenciamento 9

3.1 Estrutura de Acesso do Cluster 9

3.2 Transferindo Arquivos do Computador Local para o Cluster 9

3.3 Sempre Verifique a Integridade Após Transferir 10

3.4 Alternativa: Testando Integridade como Job do Cluster 10

4. Entendendo o Sistema de Filas do Cluster (PBS) 11

4.1 Consultando as Filas Disponíveis 11

4.2 Consultando os Limites Exatos de uma Fila 11

4.3 Filas Usadas Neste Tutorial (Exemplo Real) 12

4.4 Verificando o Tamanho da Fila Antes de Submeter 12

5. Anatomia de um Script PBS 13

5.1 Explicação de Cada Diretiva 13

5.2 Ativando o Ambiente Conda Dentro do Job 14

6. Etapa 1 — Controle de Qualidade e Trimagem 15

6.1 Concatenando Múltiplas Lanes do Mesmo Indivíduo 16

6.2 Submetendo o Job 16
6.3 Acompanhando a Execução 16
6.4 Conferindo os Resultados 17
7. Etapa 2 — Montagem do Genoma 18
7.1 O Que Esperar Durante a Execução 19
7.3 Verificando o Resultado Final 20
8. Etapa 3 — Avaliando a Qualidade da Montagem 21
8.1 QUAST — Estatísticas Estruturais da Montagem 21
8.2 BUSCO — Completude Gênica 22
9. Etapa 4 — Filtrando Contigs Muito Curtos 25
9.1 Avaliando o Impacto do Filtro Antes de Aplicá-lo 25
9.2 Aplicando o Filtro 25
10. Extras 27
10.1 Checklist de Boas Práticas antes de Submeter um Job PBS 27
10.2 Problemas Comuns 27

1. Visão Geral do Processo

Montar um genoma *de novo* significa reconstruir a sequência completa de um organismo sem usar um genoma de referência pré-existente, a partir de milhões de fragmentos curtos de DNA ("reads").

O processo é dividido em etapas sequenciais, cada uma dependente da anterior:

Etapa	O que faz?	Software
Controle de qualidade	Avalia a qualidade dos reads brutos	FastQC
Trimagem	Remove adaptadores e bases de baixa qualidade	fastp
Montagem	Reconstrói contigs/scaffolds a partir dos reads	MaSuRCA
Avaliação	Mede qualidade e completude da montagem	QUAST, BUSCO
Filtragem	Remove contigs muito curtos antes do uso	seqkit

Todo o trabalho é feito em um cluster de computação de alto desempenho (HPC) — neste caso, o ambiente Lovelace do CENAPAD-SP (Unicamp). Isto é necessário porque a montagem de um genoma de mamífero (~3 bilhões de pares de base) exige centenas de gigabytes de memória RAM e processamento paralelo, recursos que um computador convencional não tem.

1.1 Conceitos Essenciais Antes de Começar

Reads: fragmentos curtos de sequência de DNA (aqui, 150 pares de base cada), gerados pelo sequenciador Illumina. Cada indivíduo gera milhões deles.

Contig: uma sequência contínua montada a partir da sobreposição de muitos reads.

Scaffold: vários contigs unidos com lacunas (gaps) estimadas entre eles, geralmente usando informação de pareamento dos reads.

N50: métrica de qualidade da montagem. Significa que 50% do genoma montado está em contigs/scaffolds desse tamanho ou maior. Quanto maior o N50, menos fragmentada é a montagem.

Cluster HPC (High-Performance Computing): conjunto de servidores potentes compartilhados entre vários pesquisadores. O acesso e uso de recursos é controlado por um sistema de filas (neste tutorial, o PBS Pro).

Job / Script PBS: um "job" é uma tarefa computacional que você envia para a fila do cluster. Você escreve um script (`arquivo .pbs`) descrevendo o que rodar e quantos recursos (CPUs, memória, tempo) precisa, e o sistema executa quando houver disponibilidade.

1.2 Antes de Começar: O Que Você Precisa Ter

- ✓ Acesso a um cluster HPC com fila gerenciada por PBS Pro (ou similar) — neste tutorial, o Lovelace/CENAPAD-SP.
- ✓ Os arquivos de sequenciamento brutos (`.fastq.gz`), normalmente em pares R1/R2 e divididos em "lanes" (L001, L002, etc.) quando o mesmo indivíduo foi sequenciado em múltiplas lanes do sequenciador.
- ✓ Conhecimento básico de linha de comando (Linux/bash): navegar entre diretórios, copiar e mover arquivos, editar texto.
- ✓ Paciência: jobs de montagem de genoma podem rodar por dias, e filas de cluster compartilhado podem demorar horas para iniciar.

2. Preparando o Ambiente de Trabalho

Antes de rodar qualquer análise, é preciso instalar os softwares necessários e organizar os diretórios do projeto.

2.1 Introdução ao Conda

O Conda é um gerenciador de pacotes que permite a instalação de programas sem permissão do administrador. Em um cluster compartilhado, você normalmente não tem permissão para instalar softwares no sistema (isso afetaria todos os outros usuários). Desta forma, o Conda resolve isso criando "ambientes" isolados dentro da sua própria pasta pessoal (home), onde você pode instalar qualquer versão de qualquer programa sem conflitar com o resto do sistema ou com outros projetos seus.

2.2 Carregando o Conda no Cluster

A maioria dos clusters HPC usa um sistema de "módulos" para carregar softwares sob demanda. Isso evita que todos os programas fiquem disponíveis ao mesmo tempo, economizando recursos.

```
module load anaconda3
```

> ⚠ Esse comando precisa ser repetido toda vez que você precisar usar o conda. Ele não fica "salvo" entre logins.

2.3 Criando o Ambiente do Projeto

Crie um ambiente chamado `genome_assembly` contendo todos os softwares necessários para as etapas de controle de qualidade, montagem e avaliação:

```
conda create -n genome_assembly \
     --strict-channel-priority \
     -c conda-forge -c bioconda \
     python=3.11 \
     fastqc multiqc fastp \
     masurca quast busco \
     bwa samtools freebayes \
     plink admixture -y
```

Por que separamos o Python 3.11 aqui?

Algumas ferramentas de genômica de populações usadas em etapas futuras exigem versões mais antigas do Python, e são incompatíveis com os pacotes desta lista. Por isso, ferramentas com requisitos conflitantes devem ficar em ambientes conda separados!

2.4 Aceitando os Termos de Uso dos Canais (se solicitado)

Em algumas instalações recentes do Conda, é necessário aceitar os termos de uso dos canais antes de criar qualquer ambiente. Se você receber um erro mencionando "Terms of Service", rode o seguinte código:

```
conda tos accept --override-channels –channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

Depois repita o comando de criação do ambiente.

2.5 Ativando o Ambiente

Toda vez que for trabalhar no projeto, depois de carregar o módulo do Conda, é necessário ativar o ambiente novamente:

```
conda activate genome_assembly
```

Para verificar se o ambiente está ativo, procure o nome do ambiente no início da linha de comando. Neste caso, procure por “genome_assembly “.

2.6 Verificando se Tudo Foi Instalado Corretamente

Nunca assuma que a instalação funcionou sem checar. Teste sempre cada programa individualmente:

```
masurca --version bwa 2>&1 | head -3
samtools --version | head -1
fastqc --version
fastp --version 2>&1 | head -1
quast --version 2>&1 | head -1
busco --version
freebayes --version 2>&1 | head -1
```

Problema comum: versão desatualizada do samtools

O Conda pode instalar uma versão muito antiga do samtools por conflito de dependências silencioso. Isso passa despercebido porque o comando "conda install" relata sucesso sem indicar a versão. Sempre confirme a versão explicitamente. Se estiver desatualizado, force a reinstalação especificando a versão exata (código abaixo). Em seguida, confirme novamente com samtools --version. Nunca utilize --force-reinstall sem especificar a versão desejada — isso pode rebaixar (downgrade) outros pacotes do ambiente sem aviso.

```
conda install -c bioconda -c conda-forge samtools=1.23.1 -y
```

2.7 Organizando os Diretórios do Projeto

Mantenha uma estrutura de pastas clara e padronizada, isso facilita a reprodutibilidade e evita perder arquivos no meio do processo:

```
mkdir -p
~/Thrichomys/montagem_genoma/{00_raw,01_qc,02_trimmed,03_assembly,04_evaluation}
```

Pasta	Conteúdo
00_raw	Reads brutos (.fastq.gz) recebidos do sequenciamento
01_qc	Relatórios de qualidade (FastQC, fastp)
02_trimmed	Reads após trimagem de adaptadores
03_assembly	Resultado da montagem (MaSuRCA)
04_evaluation	Avaliação da montagem (QUAST, BUSCO)

3. Transferindo os Arquivos de Sequenciamento

> ⚠ Os arquivos de sequenciamento (.fastq.gz) costumam ser muito grandes (e.g. 8–10 GB por arquivo, neste estudo) e geralmente estão em um computador local antes de chegarem ao cluster. É preciso transferi-los com cuidado, pois arquivos grandes corrompidos durante a transferência geram erros difíceis de diagnosticar depois.

3.1 Estrutura de Acesso do Cluster

Muitos clusters HPC têm uma arquitetura em duas camadas: um nó "frontend" (porta de entrada, acessível via SSH) e um ou mais nós de computação/armazenamento (como o Lovelace, neste caso) acessíveis a partir do frontend. É comum que o `$HOME` do frontend seja um local diferente do `$HOME` do cluster de computação — preste atenção nisso, pois arquivos enviados para o endereço errado "desaparecem" do ponto de vista de onde você pretende trabalhar.

```
ssh -p 31459 usuario@cenapad.unicamp.br  # entra no frontend
ssh lovelace                             # do frontend, entra no Lovelace
```

3.2 Transferindo Arquivos do Computador Local para o Cluster

Use o comando `rsync` em vez de `scp` sempre que possível. O rsync verifica a integridade da transferência e permite retomar uploads interrompidos; o scp não tem essas garantias e pode silenciosamente deixar arquivos truncados em conexões instáveis.

```
rsync -avz --progress -e "ssh -p 31459" \
    MAR-T87_S14_L001_R1_001.fastq.gz \
    usuario@cenapad.unicamp.br:~/homelovelace/
```

> 💡 **Por que isso importa?**
>
> Em um caso real deste projeto, um arquivo de 9 GB foi transferido via scp e o gzip reportou corrupção ("unexpected end of file") mesmo com o tamanho do arquivo aparentemente correto. O arquivo original no computador de origem estava íntegro (confirmado com gunzip -c arquivo.gz | wc -l, que lê o arquivo por completo). A causa foi instabilidade de rede durante a transferência via scp. Repetir a transferência com rsync resolveu o problema.

3.3 Sempre Verifique a Integridade Após Transferir

Nunca assuma que um arquivo grande chegou intacto só porque o comando de transferência não mostrou erro. Teste todos os arquivos `.fastq.gz` com:

```
for f in ~/Thrichomys/montagem_genoma/00_raw/*.fastq.gz; do
  echo -n "$f: "
  gzip -t $f && echo "OK" || echo "CORROMPIDO"
done
```

O comando `gzip -t` ("test") lê o arquivo inteiro e verifica a integridade do formato comprimido, sem extrair o conteúdo. Para arquivos muito grandes, esse teste pode levar minutos — isso é esperado.

> Se o terminal cair durante um teste longo (rede instável), rode o comando como um job do cluster, em vez de interativamente. Veja a seção 3.4.

3.4 Alternativa: Testando Integridade como Job do Cluster

Quando a verificação de um arquivo grande demora muito e a conexão do terminal pode cair, submeta o teste como um job em vez de rodar interativamente:

```
#!/bin/bash
#PBS -N check_integrity
#PBS -q serial
#PBS -l select=1:ncpus=1:mem=10gb
#PBS -l walltime=04:00:00
#PBS -o /caminho/absoluto/check_integrity.out
#PBS -e /caminho/absoluto/check_integrity.err
#PBS -m abe
#PBS -M seu_email@usp.br

TRIM=/caminho/absoluto/Thrichomys/montagem_genoma/02_trimmed

gzip -t $TRIM/MAR-T87_R1_trimmed.fastq.gz
echo "R1 exit: $?"

gzip -t $TRIM/MAR-T87_R2_trimmed.fastq.gz
echo "R2 exit: $?"
```

Um "exit code" igual a 0 significa sucesso (arquivo íntegro); diferente de 0 indica problema. Submeta com `qsub nome_do_script.pbs`.

4. Entendendo o Sistema de Filas do Cluster (PBS)

Antes de rodar qualquer job, é essencial entender como o cluster organiza seus recursos em filas ("queues"). Cada fila tem limites próprios de CPUs, memória e tempo máximo de execução ("walltime"). Submeter um job para a fila errada resulta em rejeição imediata ou em desperdício de recursos.

4.1 Consultando as Filas Disponíveis

```
qstat -q
```

Esse comando lista todas as filas, com colunas indicando memória, tempo de CPU, walltime máximo, número de nós, e quantos jobs estão rodando (Run) ou esperando (Que) em cada uma.

4.2 Consultando os Limites Exatos de uma Fila

A visão geral do `qstat -q` nem sempre mostra todos os limites (alguns aparecem como "--"). Para ver os limites completos de uma fila específica, use:

```
qstat -Qf nome_da_fila | grep -E "max_|resources_"
```

> ⚠ **Erro comum: "Job violates queue and/or server resource limits"**
>
> Esse erro aparece quando você pede recursos (CPUs, memória, nós) fora do intervalo permitido pela fila. A solução é sempre consultar resources_min e resources_max da fila com qstat -Qf antes de escrever o script — nunca "adivinhar" os valores.

4.3 Filas Usadas Neste Tutorial (Exemplo Real)

Fila	CPUs	Memória	Walltime máx.	Uso recomendado
serial	1	10gb	240h	Tarefas leves e rápidas (testes, verificações)
par16	16 (fixo)	60gb	240h	FastQC, fastp, QUAST, BUSCO
par128	128 (fixo)	—	168h	Tarefas paralelas pesadas sem exigência de memória extrema
memshort	128 (fixo)	1975gb (fixo)	48h	Montagem de genoma (MaSuRCA) — alta memória, curto prazo
memlong	128 (fixo)	1975gb (fixo)	360h	Mesma finalidade de memshort, para jobs mais longos
paralela	mín. 256, 2+ nós	—	72h	Jobs MPI de larga escala (não usada neste tutorial)
serial	1	10gb	240h	Tarefas leves e rápidas (testes, verificações)



Por que não usamos a fila "paralela"?

Essa fila exige no mínimo 256 CPUs distribuídas em pelo menos 2 nós — é voltada a simulações MPI de larga escala (física, química computacional), não a pipelines de bioinformática que rodam em um único nó. Tentar submeter um job de FastQC nessa fila resulta em rejeição imediata.

4.4 Verificando o Tamanho da Fila Antes de Submeter

Filas muito concorridas podem ter dezenas de jobs esperando, atrasando o início do seu trabalho por horas. Antes de decidir em qual fila submeter, compare:

```
qstat par16  | wc -l   # conta jobs na fila par16
qstat par128 | wc -l   # conta jobs na fila par128
```

Escolha a fila compatível com seus requisitos de recursos que também tiver menor concorrência no momento.

5. Anatomia de um Script PBS

Um script PBS é um arquivo de texto com extensão `.pbs` (ou `.sh`) que combina diretivas de configuração (linhas começando com `#PBS`) e comandos normais de shell que serão executados quando o job iniciar.

```
#!/bin/bash
#PBS -N nome_do_job
#PBS -q nome_da_fila
#PBS -l select=1:ncpus=16:mem=60gb
#PBS -l walltime=12:00:00
#PBS -o /caminho/absoluto/saida.out
#PBS -e /caminho/absoluto/erro.err
#PBS -m abe
#PBS -M seu_email@usp.br

# A partir daqui, comandos normais de shell
export PATH=/caminho/do/ambiente/conda/bin:$PATH
comando_do_programa --opcoes
```

5.1 Explicação de Cada Diretiva

Diretiva	Significado
-N	Nome do job, exibido nas listagens de fila
-q	Fila onde o job será submetido
-l select=1:ncpus=N:mem=Xgb	Recursos solicitados: 1 nó, N CPUs, X GB de memória
-l walltime=HH:MM:SS	Tempo máximo de execução; o job é encerrado à força ao atingir esse limite

⚠

Problema comum: "Post job file processing error"

O sistema PBS Pro frequentemente NÃO expande o caractere ~ (atalho para a pasta pessoal) dentro das diretivas #PBS -o e #PBS -e. Isso causa um erro de pós-processamento que impede a geração correta dos logs e, em alguns casos, interrompe o envio de notificações por e-mail — mesmo que o job em si tenha rodado e produzido resultados corretamente.

ERRADO: #PBS -o ~/Thrichomys/montagem_genoma/01_qc/saida.out

CORRETO: #PBS -o /home/lovelace/proj/proj1132/usuario/Thrichomys/montagem_genoma/01_qc/saida.out

Regra prática: nunca use ~ nas diretivas #PBS -o e #PBS -e. Sempre escreva o caminho absoluto completo. Dentro do corpo do script (comandos normais de shell), o ~ funciona normalmente.

5.2 Ativando o Ambiente Conda Dentro do Job

Em muitos clusters, o comando `conda activate` não funciona corretamente dentro de jobs submetidos (os nós de computação rodam um shell não-interativo, que não carrega a configuração necessária). A solução robusta e portátil é adicionar o caminho dos binários do ambiente diretamente à variável PATH:

```
export
PATH=/home/lovelace/proj/proj1132/usuario/.conda/cenapad_miniconda3/envs/genome_assem
bly/bin:$PATH
```

Essa abordagem invoca os programas pelo caminho completo, sem depender da ativação do Conda. Para descobrir o caminho exato do seu ambiente, rode (com o ambiente já ativado interativamente):

```
echo $CONDA_PREFIX
```

Erro comum: "Process Substitution" (<(...)) em Scripts PBS

A sintaxe <(comando), chamada "process substitution", funciona no terminal interativo (bash) mas pode falhar em shells não-interativos usados pelo PBS para executar jobs — o programa recebe um arquivo vazio sem reportar erro. Isso já causou arquivos de saída de 0 reads neste projeto ("total_reads": 0 no relatório do fastp), sem qualquer mensagem de erro visível.

ERRADO (falha silenciosa em jobs PBS):

fastp --in1 <(cat lane1.fastq.gz lane2.fastq.gz) ...

CORRETO (gera um arquivo intermediário primeiro):

cat lane1.fastq.gz lane2.fastq.gz > concatenado.fastq.gz

fastp --in1 concatenado.fastq.gz ...

Regra prática: sempre que precisar concatenar ou combinar arquivos antes de um programa, faça isso em um passo separado, gerando um arquivo intermediário real, e SEMPRE confira o tamanho e conteúdo do resultado antes de seguir para a próxima etapa.

6. Etapa 1 — Controle de Qualidade e Trimagem

Esta etapa avalia a qualidade dos reads brutos e remove adaptadores de sequenciamento e bases de baixa qualidade, preparando os dados para a montagem.

FastQC: gera um relatório visual da qualidade de cada arquivo de reads — distribuição de qualidade por posição, conteúdo de GC, presença de adaptadores, duplicação de sequências. É um diagnóstico, não modifica os dados.

fastp: remove adaptadores de sequenciamento (sequências artificiais inseridas durante a preparação da biblioteca) e corta ou descarta bases/reads de baixa qualidade. Reads com adaptadores ou erros de leitura não removidos geram montagens fragmentadas e erros falsos na chamada de variantes (SNP calling) mais adiante.

Script PBS Completo:

```
#!/bin/bash
#PBS -N fastqc_fastp
#PBS -q par16
#PBS -l select=1:ncpus=16:mem=60gb
#PBS -l walltime=48:00:00
#PBS -o /caminho/absoluto/Thrichomys/montagem_genoma/01_qc/fastqc_fastp.out
#PBS -e /caminho/absoluto/Thrichomys/montagem_genoma/01_qc/fastqc_fastp.err
#PBS -m abe
#PBS -M seu_email@usp.br

# Ambiente
export
PATH=/caminho/absoluto/.conda/cenapad_miniconda3/envs/genome_assembly/bin:$PATH

# Diretórios
RAW=/caminho/absoluto/Thrichomys/montagem_genoma/00_raw
QC=/caminho/absoluto/Thrichomys/montagem_genoma/01_qc
TRIM=/caminho/absoluto/Thrichomys/montagem_genoma/02_trimmed
cd $RAW

# FastQC nos reads brutos (todos os lanes, R1 e R2)
fastqc -t 16 -o $QC \
  AMOSTRA_L001_R1_001.fastq.gz AMOSTRA_L001_R2_001.fastq.gz \
  AMOSTRA_L002_R1_001.fastq.gz AMOSTRA_L002_R2_001.fastq.gz \
  AMOSTRA_L003_R1_001.fastq.gz AMOSTRA_L003_R2_001.fastq.gz \
  AMOSTRA_L004_R1_001.fastq.gz AMOSTRA_L004_R2_001.fastq.gz

# Concatenar os 4 lanes em arquivos intermediarios
# (NUNCA use process substitution <(...) aqui --- ver secao 5.4)
cat AMOSTRA_L001_R1_001.fastq.gz AMOSTRA_L002_R1_001.fastq.gz \
  AMOSTRA_L003_R1_001.fastq.gz AMOSTRA_L004_R1_001.fastq.gz \
  > $TRIM/AMOSTRA_R1_concat.fastq.gz
```

```
cat AMOSTRA_L001_R2_001.fastq.gz AMOSTRA_L002_R2_001.fastq.gz \
  AMOSTRA_L003_R2_001.fastq.gz AMOSTRA_L004_R2_001.fastq.gz \
  > $TRIM/AMOSTRA_R2_concat.fastq.gz

# fastp: trimagem dos reads concatenados
fastp \
  --in1 $TRIM/AMOSTRA_R1_concat.fastq.gz \
  --in2 $TRIM/AMOSTRA_R2_concat.fastq.gz \
  --out1 $TRIM/AMOSTRA_R1_trimmed.fastq.gz \
  --out2 $TRIM/AMOSTRA_R2_trimmed.fastq.gz \
  --detect_adapter_for_pe \
  --thread 16 \
  --html $QC/AMOSTRA_fastp.html \
  --json $QC/AMOSTRA_fastp.json \
  --report_title "AMOSTRA"

# Remover arquivos concatenados intermediarios (economizar espaco)
rm $TRIM/AMOSTRA_R1_concat.fastq.gz $TRIM/AMOSTRA_R2_concat.fastq.gz

echo "FastQC + fastp concluidos: $(date)"
```



Substitua AMOSTRA pelo nome real da sua amostra e /caminho/absoluto/ pelo caminho real do seu $HOME no cluster (use pwd para descobrir).

6.1 Concatenando Múltiplas Lanes do Mesmo Indivíduo

Quando um indivíduo é sequenciado em múltiplas pistas (lanes) do equipamento Illumina (L001, L002, L003, L004...), os arquivos resultantes pertencem à mesma amostra biológica e devem ser tratados como um único conjunto de reads. A prática recomendada é concatenar todas as lanes de R1 entre si, e todas as lanes de R2 entre si, antes da trimagem — nunca misturar R1 com R2.

6.2 Submetendo o Job

```
qsub fastqc_fastp.pbs
```

O comando retorna um identificador de job (ex.: 889131.ada). Guarde esse número para acompanhamento.

6.3 Acompanhando a Execução

```
qstat -u seu_usuario
```

Diretiva	Significado
Q	Queued — na fila, aguardando recursos disponíveis
R	Running — rodando
H	Held — em espera por dependência ou intervenção manual
E	Exiting — finalizando

6.4 Conferindo os Resultados

Depois que o job terminar (status some da listagem do qstat, ou chega o e-mail de "end"), verifique:

```
ls -lh ~/Thrichomys/montagem_genoma/02_trimmed/
cat ~/Thrichomys/montagem_genoma/01_qc/AMOSTRA_fastp.json | grep -E
"total_reads|total_bases|q30_rate"
```



Problema comum: "total_reads": 0

Se o relatório do fastp mostrar "total_reads": 0, os arquivos trimados estão vazios e NÃO podem ser usados na montagem — mesmo que o job tenha terminado sem mensagem de erro explícita. A causa mais provável é o uso de “process substitution”, descrito na seção 5.2.

Confira também o tamanho dos arquivos com ls -lh; arquivos de poucos bytes (ex. 23 bytes) confirmam o problema.

7. Etapa 2 — Montagem do Genoma

Esta é a etapa central de todo o processo: o MaSuRCA reconstrói o genoma a partir dos reads trimados, gerando contigs e scaffolds.

Script PBS Completo:

```
#!/bin/bash
#PBS -N masurca_assembly
#PBS -q memshort
#PBS -l select=1:ncpus=128:mem=1975gb
#PBS -l walltime=48:00:00
#PBS -o /caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/masurca.out
#PBS -e /caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/masurca.err
#PBS -m abe
#PBS -M seu_email@usp.br

export
PATH=/caminho/absoluto/.conda/cenapad_miniconda3/envs/genome_assembly/bin:$PATH
TRIM=/caminho/absoluto/Thrichomys/montagem_genoma/02_trimmed
ASSEMBLY=/caminho/absoluto/Thrichomys/montagem_genoma/03_assembly
cd $ASSEMBLY

# Gerar o script de montagem do MaSuRCA
masurca \
  -i $TRIM/AMOSTRA_R1_trimmed.fastq.gz,$TRIM/AMOSTRA_R2_trimmed.fastq.gz \
  -t 128 \
  -o assemble.sh

# Executar a montagem
bash assemble.sh

echo "MaSuRCA concluido: $(date)"
```



Escolhendo a Fila Correta

Consulte os limites antes de escrever o script (ver seção 4.2). Para este projeto, a fila memshort exige exatamente 128 CPUs e 1975 GB de memória, com até 48h de walltime.

```
qstat -Qf memshort | grep -E "resources_min|resources_max"
```

Por Que a Montagem Exige Tanta Memória?

O MaSuRCA constrói estruturas de dados internas (k-mers, grafos de sobreposição) que crescem proporcionalmente ao tamanho do genoma e à profundidade de cobertura. Um genoma de mamífero de aproximadamente 3 Gb sequenciado a 60x de cobertura pode precisar de mais de 1 TB de memória RAM durante os picos do processo. Isso explica por que a montagem precisa rodar em filas específicas de "alta memória" do cluster.

7.1 O Que Esperar Durante a Execução

O MaSuRCA passa por várias etapas internas sequenciais: contagem de k-mers, construção de super reads, geração de unitigs (4-unitigger), consenso (5-consensus), scaffolding e fechamento de gaps (gapclose). O processo completo, para um genoma de mamífero, costuma levar entre 24 e 48 horas mesmo com recursos abundantes.

Para acompanhar o progresso sem esperar o job terminar, observe os arquivos sendo gerados na pasta CA, ordenados por data de modificação:

```
ls -lt ~/Thrichomys/montagem_genoma/03_assembly/CA/ | head -10
```

Arquivos recentes indicam que o processo está ativo. O aparecimento da pasta 5-consensus, seguido por arquivos como scaffolds.ref.fa e finalmente primary.genome.scf.fasta, indica avanço nas etapas finais.

Estouro de Walltime: "Stop Requested Before 'Scaffolder'"

Se o job for encerrado pelo sistema antes de concluir (mensagem "PBS: job killed: walltime exceeded limit" no arquivo .err, ou "Stop requested before..." no log do MaSuRCA), o trabalho NÃO está necessariamente perdido. O MaSuRCA salva o progresso em disco a cada etapa concluída.

Antes de tomar qualquer decisão, verifique exatamente em qual etapa o processo parou:

`ls -lt ~/Thrichomys/montagem_genoma/03_assembly/CA/ | head -10`

Se a pasta de trabalho ainda contiver os arquivos intermediários das etapas anteriores, é possível retomar a partir do ponto onde parou, simplesmente rodando novamente `bash assemble.sh` — o MaSuRCA detecta o progresso já salvo e não refaz etapas concluídas.

Script de Retomada:

```
#!/bin/bash
#PBS -N masurca_resume
#PBS -q memshort
#PBS -l select=1:ncpus=128:mem=1975gb
#PBS -l walltime=48:00:00
#PBS -o /caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/masurca_resume.out
#PBS -e /caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/masurca_resume.err
#PBS -m abe
#PBS -M seu_email@usp.br

export
PATH=/caminho/absoluto/.conda/cenapad_miniconda3/envs/genome_assembly/bin:$PATH
ASSEMBLY=/caminho/absoluto/Thrichomys/montagem_genoma/03_assembly
cd $ASSEMBLY

# Retomar a partir do ponto salvo
bash assemble.sh

echo "MaSuRCA retomado e concluido: $(date)"
```



Erro ao Resubmeter: "Would exceed queue generic's per-user limit"

Filas de altíssima memória (como memshort e memlong) costumam permitir apenas 1 job por usuário simultaneamente. Se você receber esse erro ao tentar submeter o job de retomada, verifique antes se o job anterior já terminou de fato:

`qstat -u seu_usuario`

Se o job anterior ainda aparecer com status R (running), ele ainda não terminou — aguarde sua conclusão (ou o encerramento por walltime) antes de submeter a retomada.

7.3 Verificando o Resultado Final

O arquivo final da montagem chama-se `primary.genome.scf.fasta`, dentro da pasta CA. Confirme sua existência e conte quantos contigs/scaffolds foram gerados:

```
grep -c ">" ~/Thrichomys/montagem_genoma/03_assembly/CA/primary.genome.scf.fasta
```

Cada sequência no formato FASTA começa com uma linha de cabeçalho marcada por ">". Contar essas linhas equivale a contar o número total de contigs/scaffolds na montagem.

8. Etapa 3 — Avaliando a Qualidade da Montagem

A montagem não está pronta só porque o programa terminou sem erro. É preciso medir objetivamente sua qualidade antes de utilizá-la como referência em análises futuras.

8.1 QUAST — Estatísticas Estruturais da Montagem

O QUAST calcula métricas como número de contigs, tamanho total, N50, conteúdo de GC e o percentual de bases ambíguas (N). Essas métricas indicam o quão fragmentada está a montagem.

```
#!/bin/bash
#PBS -N quast_eval
#PBS -q par16
#PBS -l select=1:ncpus=16:mem=60gb
#PBS -l walltime=04:00:00
#PBS -o /caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation/quast.out
#PBS -e /caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation/quast.err
#PBS -m abe
#PBS -M seu_email@usp.br

export
PATH=/caminho/absoluto/.conda/cenapad_miniconda3/envs/genome_assembly/bin:$PATH
ASSEMBLY=/caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/CA/primary.genome.s
cf.fasta
OUT=/caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation/quast_results

quast.py $ASSEMBLY \
  -o $OUT \
  -t 16 \
  --large \
  --est-ref-size 3000000000

echo "QUAST concluido: $(date)"
```

Entendendo o comando:

--large: ativa otimizações de memória e desempenho indicadas para genomas grandes (acima de ~100 Mb).

--est-ref-size: fornece uma estimativa do tamanho real do genoma (em pares de base), usada para calcular a métrica NG50, mais comparável entre montagens diferentes do que o N50 simples.

Intrepretando o relatório do QUAST:

O resultado principal fica em `quast_results/report.txt`:

```
cat ~/Thrichomys/montagem_genoma/04_evaluation/quast_results/report.txt
```

Métrica	O que indica
# contigs	Quanto menor, menos fragmentada a montagem
N50	50% do genoma está em contigs deste tamanho ou maior; quanto maior, melhor
Total length	Soma do tamanho de todos os contigs; deve se aproximar do tamanho esperado do genoma
GC (%)	Composição de bases; útil para comparar com valores esperados da espécie
# N's per 100 kbp	Frequência de bases ambíguas (gaps); quanto menor, melhor

8.2 BUSCO — Completude Gênica

Enquanto o QUAST avalia a estrutura da montagem, o BUSCO avalia seu conteúdo biológico. Ele busca por um conjunto de genes universalmente conservados em um grupo taxonômico (neste caso, roedores) e reporta quantos foram encontrados completos, fragmentados ou ausentes.

```
#!/bin/bash
#PBS -N busco_eval
#PBS -q par16
#PBS -l select=1:ncpus=16:mem=60gb
#PBS -l walltime=24:00:00
#PBS -o /caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation/busco.out
#PBS -e /caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation/busco.err
#PBS -m abe
#PBS -M seu_email@usp.br

export
PATH=/caminho/absoluto/.conda/cenapad_miniconda3/envs/genome_assembly/bin:$PATH
ASSEMBLY=/caminho/absoluto/Thrichomys/montagem_genoma/03_assembly/CA/primary.genome.s
cf.fasta
OUT=/caminho/absoluto/Thrichomys/montagem_genoma/04_evaluation
cd $OUT

busco \
  -i $ASSEMBLY \
  -o busco_results \
  -m genome \
```

```
  -l rodentia_odb12 \
  -c 16 \
  --force

echo "BUSCO concluido: $(date)"
```

Entendendo o comando:

-l rodentia_odb12: especifica o banco de dados de genes conservados a ser usado — deve corresponder ao grupo taxonômico do organismo estudado. Trocar para o banco taxonômico correto da espécie de interesse é essencial; bancos genéricos (ex. eukaryota) dão resultados pouco informativos.

Conectividade do nó de computação

Na primeira execução, o BUSCO precisa baixar o banco de dados de genes (lineage) da internet. Em alguns clusters, os nós de computação não têm acesso à internet, e o download falhará. Verifique se o banco já foi baixado previamente:

`find ~ -iname "*busco_downloads*" 2>/dev/null`

Se a pasta existir e contiver o banco desejado (ex. rodentia_odb12), o job pode rodar mesmo sem conectividade nova. Caso contrário, baixe o banco manualmente a partir do nó frontend (que normalmente tem acesso à internet) antes de submeter o job.

Intrepretando o relatório do BUSCO:

O resultado resumido aparece em `busco_results/short_summary*.txt`, em uma linha no formato:

```
C:76.6%[S:74.0%,D:2.6%],F:12.9%,M:10.5%,n:12639
```

Sigla	Significado
C (Complete)	Genes encontrados completos — soma de single-copy e duplicated
S (Single-copy)	Genes completos encontrados em uma única cópia (o esperado para a maioria)
D (Duplicated)	Genes completos encontrados em mais de uma cópia (pode indicar duplicação genômica real ou erro de montagem)
F (Fragmented)	Genes encontrados apenas parcialmente — geralmente reflexo de fragmentação da montagem

Sigla	Significado
M (Missing)	Genes esperados que não foram encontrados
n	Número total de genes buscados no banco de dados
C (Complete)	Genes encontrados completos — soma de single-copy e duplicated



Como interpretar o percentual de completude

Para montagens de genomas de mamíferos feitas exclusivamente com reads curtos Illumina, sem tecnologias de leitura longa complementares, uma completude (C) entre 70% e 85% é considerada normal e adequada para análises de genômica de populações baseadas em SNPs (*e. g.* estrutura populacional, PCA, ADMIXTURE, delimitação de espécies).

Valores muito abaixo disso (*e.g.* <10%) geralmente indicam um problema na execução, não na biologia do organismo. Portanto, devem ser investigados antes de aceitar o resultado.

9. Etapa 4 — Filtrando Contigs Muito Curtos

Antes de utilizar a montagem como referência para mapear outros indivíduos, é prática padrão remover contigs muito curtos.

Por que filtrar Contigs curtos?

- Contigs muito curtos (poucas centenas de pares de base) costumam ser ruído de montagem: repetições mal resolvidas, fragmentos de contaminação, ou pedaços que não conseguiram se unir a contigs maiores.
- Reads que mapeiam nesses contigs tendem a ter qualidade de mapeamento baixa (multi-mapping), pois são sequências curtas e pouco específicas, inflando estatísticas sem gerar SNPs confiáveis.
- Manter milhares de contigs irrelevantes aumenta o tempo de processamento de etapas seguintes (indexação, mapeamento, chamada de variantes) sem ganho real de informação.

9.1 Avaliando o Impacto do Filtro Antes de Aplicá-lo

Nunca aplique um filtro de tamanho "no escuro". Primeiro, verifique quantos contigs e quantos pares de base seriam removidos com o corte escolhido (aqui, 500 pb), e compare com o total da montagem:

```
seqkit fx2tab -nl primary.genome.scf.fasta | \
  awk '{if ($2 < 500) {n++; bp+=$2}} END {print "Contigs <500bp:", n; print "Total bp
perdido:", bp}'
```

Compare o "bp perdido" com o tamanho total da montagem (visto no relatório do QUAST). Se a perda for uma fração muito pequena do total (no estudo de caso deste tutorial, 0,21% do genoma, mesmo removendo 21,5% dos contigs), o filtro está bem calibrado: remove principalmente ruído, sem sacrificar cobertura genômica real.

9.2 Aplicando o Filtro

```
seqkit seq -m 500 primary.genome.scf.fasta \
  -o primary.genome.scf.filtered.fasta
```

Entendendo o comando:

-m 500: mantém apenas sequências com 500 pares de base ou mais.

Confirme o resultado:

```
grep -c ">" primary.genome.scf.filtered.fasta
```

O arquivo `primary.genome.scf.filtered.fasta` é a versão final da montagem, pronta para ser usada como referência de mapeamento nas etapas seguintes da pipeline (indexação com BWA, mapeamento de outros indivíduos, chamada de variantes).

10. Extras

10.1 Checklist de Boas Práticas antes de Submeter um Job PBS

1. Os caminhos nas diretivas `#PBS -o` e `#PBS -e` são absolutos (sem ~)?

2. A fila escolhida tem CPUs, memória e walltime compatíveis com o que o script pede? (confirme com `qstat -Qf`)

3. O ambiente Conda está sendo ativado via `export PATH` (não via `conda activate`)?

4. O script evita process substitution (`<(...)`)? Concatenações foram feitas em etapas separadas com arquivos intermediários?

5. As diretivas de e-mail (`-m abe -M seu_email`) estão presentes para notificação de início, fim e erro?

6. Há espaço em disco suficiente para arquivos intermediários temporários? (confira com `df -h ~`)

10.2 Problemas Comuns

Sintoma	Causa Provável	Solução
"Job violates queue and/or server resource limits"	Recursos pedidos fora do permitido pela fila	Consulte `qstat -Qf nome_da_fila` e ajuste `ncpus/mem/walltime`
"Post job file processing error"	Uso de ~ nas diretivas -o/-e	Use caminho absoluto completo nessas diretivas
Relatório mostra total_reads: 0	Process substitution falhou silenciosamente	Substitua `<(cat ...)` por concatenação em arquivo intermediário
gzip -t retorna código 1 (corrompido)	Falha durante transferência de arquivo	Retransferir usando rsync em vez de scp
"would exceed queue generic's per-user limit"	Fila de uso exclusivo já tem um job seu rodando	Aguarde o job anterior terminar antes de submeter outro
BUSCO com completude muito baixa (<10%)	Modo de execução inadequado ou tentativa de download sem internet no nó	Verifique o modo usado no log e a presença do banco em `busco_downloads/`
samtools com versão muito antiga após instalação	Conflito de dependências silencioso do Conda	Reinstale especificando a versão exata: samtools=1.23.1

Princípio Geral do Processo: Confie, mas verifique

Um job terminar sem mensagem de erro não significa que o resultado está correto. Em praticamente todos os problemas encontrados neste tutorial — arquivo corrompido, reads vazios, montagem incompleta — o job "terminou" sem alarme explícito.

O hábito que realmente evita retrabalho é: depois de cada etapa, confira o tamanho dos arquivos gerados, conte registros quando possível (reads, contigs), e compare com valores esperados antes de avançar para a etapa seguinte.